

Lenguaje SQL

[Descargar estos apuntes](#)

Índice

▼ Lenguaje SQL

- Lenguaje de Manipulación de Datos

- ▼ Consultas `SELECT` básicas

- Consultas con operaciones numéricas y literales
- Consultas con funciones predefinidas de información
- Consultas básicas a tablas

- ▼ Utilidad de captura

- Ejemplos de las primeras consultas
- Consultas con selección de registros
- Consultas con varias condiciones
- Consultas con funciones

- ▼ Consultas `SELECT` de AGRUPACIÓN

- Consultas de agrupación
- Ejemplos de las primeras consultas agrupadas

- ▼ Consultas `SELECT` multitabla

- Consultas multitabla de cruce (relación 1-N)
- Consultas multitabla mediante `INTERSECCIÓN`
- Consultas multitabla mediante `UNION`

- ▼ Consultas multitabla mediante `SUBCONSULTAS`

- Subconsultas en `WHERE` con operador de comparación
- Subconsultas en `WHERE` con operador `IN`
- Subconsultas en `WHERE` con operador `EXISTS`
- Subconsultas en `FROM`

Lenguaje de Manipulación de Datos

El **Lenguaje de Manipulación de Datos** (**LMD**, en inglés Data Manipulation Language, **DML**) es un lenguaje proporcionado por el sistema de gestión de base de datos que permite a los usuarios de la misma llevar a cabo las tareas de consulta o manipulación de los datos (inserción, borrado, actualización y consultas) basado en el modelo de datos adecuado.



Las **consultas** recuperan información de las tablas de una base de datos mediante la selección de campos, la realización de filtros y la transformación de los datos recuperados.

El uso de consultas permite:

- **Elegir tablas.** Se puede obtener información de una sola tabla o de varias.
- **Elegir campos.** Se pueden especificar los campos a visualizar de cada tabla.
- **Elegir registros.** Se pueden seleccionar los registros a mostrar en la hoja de respuestas dinámica, especificando un criterio.
- **Ordenar registros.** Se puede ordenar la información en forma ascendente o descendente.
- **Realizar cálculos.** Se pueden emplear las consultas para hacer cálculos con los datos de las tablas.

Además, en consultas más complejas se puede:

- **Crear tablas.** Se puede generar otra tabla a partir de los datos combinados de una consulta. La tabla se genera a partir de la hoja de respuestas dinámica.
- **Consultas encadenadas.** Utilizar una consulta como origen de datos para otras consultas (subconsulta). Se pueden crear consultas adicionales basadas en un conjunto de registros seleccionados por una consulta anterior.

Consultas **SELECT** básicas

Consultas con operaciones numéricas y literales

Comenzaremos por utilizar la instrucción **SELECT** como si fuera una calculadora. Para ello utilizaremos los siguientes operadores aritméticos:

Operadores aritméticos

Operador	Función
+	Suma
-	Resta
*	Producto o Multiplicación
/	División
DIV	División entera
MOD o %	Resto de la división

Los siguientes ejemplos evalúan las expresiones que hay después de la cláusula **SELECT** y muestran el resultado por pantalla.

Ejemplo 1

La última sentencia muestra tres columnas.

```
-- Salida: 3000
SELECT 20 * 150;

-- Salida: 14.5
SELECT ((5 * 4.5) / 3) + 7;

-- Salida: 33.33333333, 33, 1
SELECT 100 / 3, 100 DIV 3, 100 MOD 3;
```

Ejemplo 2

Para el uso de literales o cadenas de caracteres basta con colocar unas comillas.

```
-- Salida: Felicidades
SELECT 'Felicidades';

-- Salida: Precio, 25 * 1.21 =, 30.25
SELECT 'Precio', '25 * 1.21 =', 25 * 1.21;
```

Referencias

[MySQL – Operadores aritméticos](#)

[MySQL Oficial - Operadores](#)

Consultas con funciones predefinidas de información

A través de funciones ya predefinidas en el MySQL podemos obtener información sobre diferentes aspectos del estado de nuestra conexión y del sistema. Algunos ejemplos son:

Ejemplo 1

La **versión** de MySQL y nuestro identificador (**ID**) de conexión.

```
SELECT version(), connection_id();
```

Ejemplo 2

El usuario actual con el que estamos conectados y la base de datos seleccionada.

```
SELECT user(), database();
```

Ejemplo 3

Fecha y hora (`now()`) o sólo la fecha `current_date()`) del sistema.

```
SELECT now(), current_date();
```



Referencias

[MySQL – Funciones de información](#)

[MySQL Oficial – Funciones de información](#)

Consultas básicas a tablas

```
SELECT [DISTINCT] campos  
[FROM table_references]  
[WHERE where_definition]  
[ORDER BY {col_name | expr | position} [ASC | DESC] , ...]  
[LIMIT [offset,] row_count]
```



Importante

El orden de las cláusulas, **obligatoriamente**, es:

SELECT ... FROM ... WHERE ... ORDER ... LIMIT

Utilidad de captura

En muchas ocasiones nos interesaría llevar un registro de las instrucciones SQL ejecutadas y su resultado. Conectados con la utilidad `mysql.exe` a MySQL podemos grabar toda la salida a un archivo a través de un comando:

```
mysql> tee archivo.txt
mysql> ... instrucciones SQL ...**
```

Para terminar la salida:

```
mysql> notee
```

Ejemplos de las primeras consultas

Con XAMPP-MySQL importar la BD de jardinería que facilitará el profesor.

Ejemplo 1

Tablas completas

Mostrar todos los campos de todos los clientes.

```
SELECT *
FROM clientes;
```

Ejemplo 2

Seleccionar campos a mostrar

Mostrar los campos `CodigoCliente`, `NombreCliente`, `Telefono`, `Ciudad`, `Region`, `Pais` de todos los clientes.

```
SELECT CodigoCliente, NombreCliente, Telefono, Ciudad, Region, Pais
FROM clientes;
```

Ejemplo 3

Campos calculados

Mostrar los campos `CodigoCliente`, `NombreCliente`, `LimiteCredito` de todos los clientes añadiendo un campo calculado que es el `LimiteCreditoMensual` como `LimiteCredito / 12`. Haced la division entera para evitar decimales. Utilizaremos el **alias de campo** con la palabra reservada `AS`.

```
SELECT CodigoCliente, NombreCliente, LimiteCredito, LimiteCredito DIV 12 AS LimiteCreditoMensual
FROM clientes;
```

Ejemplo 4

No mostrar repetidos

Mostrar las regiones (campo **Region**) todos los clientes evitando resultandos repetidos.

```
SELECT DISTINCT Region
FROM clientes;
```

Ejemplo 5

Ordenar registros

Mostrar todos los campos de todos los clientes ordenados por **LimiteCredito** ordenado descendentemente.

```
SELECT * FROM clientes
ORDER BY LimiteCredito DESC;
```

Ejemplo 6

Limitar el número de registros a mostrar del resultado

Utilizando la consulta del ejemplo anterior mostrar:

- Sólo los 5 primeros registros

```
SELECT *
FROM clientes
ORDER BY LimiteCredito DESC
LIMIT 5;
```

- Empezando en el tercer registro, mostrar 10 registros

```
SELECT *
FROM clientes
ORDER BY LimiteCredito DESC
LIMIT 2,10;
```

Consultas con selección de registros

Para poder seleccionar registros es necesario indicar la condición que deben cumplir los campos de un registro para ser mostrado. Para ello se utiliza la sección **WHERE** de la instrucción SQL.

Los operadores relacionales que nos permiten comparar el valor de los campos son:

Operadores relacionales

Operador	Función
=	Igual a
<	Menor que
>	Mayor que
<=	Menor o igual
>=	Mayor o igual
LIKE	Patrón que debe cumplir un campo cadena
BETWEEN	Intervalo de valores
IN	Conjunto de valores
IS NULL	Es nulo
IS NOT NULL	No es nulo

Veamos unos cuantos ejemplos:

Ejemplo 1

Buscar un registro por el valor de su clave

Mostrar todos los campos del cliente con código igual a 12.

```
SELECT *  
FROM clientes  
WHERE CodigoCliente = 12;
```

 Al ser el campo clave, el resultado sólo mostrará un registro.

Ejemplo 2

Buscar registros por el valor de un campo

Mostrar todos los campos de los clientes de la **Región** Madrid.

```
SELECT *  
FROM clientes  
WHERE Region = 'Madrid';
```

 Al NO ser un campo clave, el resultado puede mostrar más de un registro.

Ejemplo 3

Buscar registros por comparación del valor de un campo

Mostrar todos los campos de los clientes con Límite de Crédito mayor de 50000€.

```
SELECT *  
FROM clientes  
WHERE LimiteCredito > 50000;
```

Ejemplo 4

Buscar registros por comparación del valor de un campo

Mostrar todos los campos de los clientes con Límite de Crédito entre 10000€ y 60000€.

```
SELECT *  
FROM clientes  
WHERE LimiteCredito BETWEEN 10000 AND 60000;
```

Ejemplo 5

Buscar registros por comparación del valor de un campo

Mostrar todos los campos de los clientes con la **Región** nula.

```
SELECT *  
FROM clientes  
WHERE Region IS NULL;
```

Ejemplo 6

Ejemplo combinando con el ejemplo anterior

Mostrar los Países de los clientes con la **Región** nula, sin repetir valores.

```
SELECT DISTINCT Pais  
FROM clientes  
WHERE Region IS NULL;
```

Ejemplo 7

Ejemplo con el operador `LIKE`

Mostrar los Empleados cuyo `email` contenga *jardin*.

```
SELECT *  
FROM Empleados  
WHERE Email LIKE '%jardin%';
```

 `%` es una cadena de caracteres cualquiera y `_` es un sólo carácter cualquiera.

Ejemplo 8

Ejemplo con el operador `IN`

Mostrar los productos de las gamas *Aromáticas*, *Herramientas* y *Ornamentales*.

```
SELECT *  
FROM productos  
WHERE Gama IN ('Aromáticas', 'Herramientas', 'Ornamentales');
```

Referencias

[MySQL – Consultas](#)

[MySQL – Operadores de comparación](#)

[MySQL Oficial – Operadores de comparación](#)

Consultas con varias condiciones

Para poder realizar varias condiciones necesitamos combinarlas con operadores lógicos, que en MySQL son:

Operadores lógicos

Operador	Función
AND	Y
OR	O
NOT	No

Veamos unos cuantos ejemplos:

Ejemplo 1

Ejemplo con varias condiciones

Mostrar código (`CodigoProducto`) y `nombre` de los Productos que sean de la `Gama` *Frutales* y `CantidadEnStock` sea mayor que 50 unidades. Mostremos también los campos implicados en las condiciones para comprobar el resultado.

```
SELECT CodigoProducto, Nombre, Gama,  
FROM productos  
WHERE Gama = 'Frutales' AND CantidadEnStock > 50;
```

Ejemplo 2

Ejemplo con varias condiciones

Mostrar código y nombre de los Clientes que sean de la `Ciudad` *Madrid* o *Barcelona*. Mostremos también los campos implicados en las condiciones para comprobar el resultado.

```
SELECT CodigoCliente, NombreCliente, Ciudad  
FROM clientes  
WHERE Ciudad = 'Madrid' OR Ciudad = 'Barcelona';
```

Referencias

[MySQL – Consultas y operadores de comparación \(curso\)](#)

[MySQL Oficial – Operadores lógicos](#)

Consultas con funciones

Las funciones nos permiten realizar transformaciones de los datos para obtener información. Existen multitud de funciones que procesan diferentes tipos de datos.

Funciones de cadenas de caracteres

Función	Descripción
CONCAT(cad1, cad2, ...)	Concatena cadenas
UPPER(cad)	Pasa a mayúsculas una cadena
LOWER(cad)	Pasa a minúsculas una cadena
LTRIM(cad)	Elimina de la cadena los espacios iniciales y finales
LEFT(cad, X)	Obtiene los X primeros caracteres de la cadena
RIGHT(cad, X)	Obtiene los X últimos caracteres de la cadena
LENGTH(cad)	Longitud de una cadena
REPLACE(cad, ant, pos)	Obtiene una cadena tomando <i>cad</i> como origen y cambiando <i>ant</i> por <i>pos</i>

Funciones numéricas de un campo

Función	Descripción
RAND()	Número aleatorio decimal entre 0 y 1 (menor que 1)
POW(num, exp)	Obtiene la potencia <i>num</i> elevado a <i>exp</i>
FLOOR(num)	Obtiene la parte entera de un número decimal
ROUND(num, X)	Redondea un número a X decimales
SIGN(num)	Devuelve 1 para positivo, 0 para 0 y -1 para negativo
ABS(num)	Obtiene el valor absoluto de un número

Funciones de fechas de un campo

Función	Descripción
YEAR(campo)	Muestra el año del valor de un campo de tipo fecha
MONTH(campo)	Muestra el mes del valor de un campo de tipo fecha
DAY(campo)	Muestra el día del valor de un campo de tipo fecha
DAYNAME(campo)	Muestra el nombre del día de la semana del campo
DAYOFWEEK(campo)	Devuelve un número indicando el día de la semana de un campo fecha: <i>1=Domingo, 2=Lunes, 3=Martes, 4=Miércoles, 5=Jueves, 6=Viernes, 7=Sábado.</i>

Funciones numéricas con varios registros

Función	Descripción
COUNT(*)	Cuenta los registros seleccionados
MIN(campo)	Valor mínimo del campo de los registros seleccionados
MAX(campo)	Valor máximo del campo de los registros seleccionados
SUM(campo)	Suma de los valores del campo
AVG(campo)	Media de los valores del campo

Referencias

[MySQL w3schools – Funciones de MySQL](#)

[MySQL Oficial – Funciones](#)

Funciones para campos calculados y para condiciones

Ejemplo 1

Ejemplo con funciones

Mostrar código (`CodigoProducto`), `nombre` y `precio de venta al público` de los productos, pero ese precio debe ser con IVA incluido, es decir, agregarle al `PrecioVenta` el 21% multiplicándolo por 1.21. Asignar el alias `pvp` al nuevo campo calculado.

```
SELECT CodigoProducto, Nombre, PrecioVenta, ROUND(PrecioVenta * 1.21, 2) AS pvp
FROM Productos;
```

Ejemplo 2

Ejemplo con funciones en las condiciones

Seleccionar en la consulta anterior los productos que tengan el nuevo campo **pvp** mayor de 100€.

```
SELECT CodigoProducto, Nombre, PrecioVenta, ROUND(PrecioVenta * 1.21, 2) AS pvp
FROM Productos
WHERE ROUND(PrecioVenta * 1.21, 2) > 100;
```

Funciones para manejo de cadenas de caracteres

Ejemplo 3

Ejemplo con funciones

Seleccionar de los empleados el nombre completo (**NombreCompleto**) concatenando el nombre y los dos apellidos.

```
SELECT CONCAT(Nombre, ' ', Apellido1, ' ', Apellido2) AS NombreCompleto
FROM empleados;
```

Ejemplo 4

Ejemplo con funciones

Obtener la inicial del **nombre** de todos los empleados.

```
SELECT LEFT(Nombre, 1) AS inicial
FROM empleados;
```

Ejemplo 5

Ejemplo con funciones

Obtener el **nombre** de los empleados todo en mayúsculas.

```
SELECT UCASE(Nombre)
FROM empleados;
```

Ejemplo 6

Ejemplo con funciones

Obtener el código de las oficinas (**CodigoOficina**) pero sustituyendo el guión por la barra, es decir, el '-' por '/'.

```
SELECT REPLACE(CodigoOficina, '-', '/')
FROM oficinas;
```

Ejemplo 7

Ejemplo con funciones

Obtener la primera palabra del nombre del **proveedor** de todos los productos.

```
SELECT SUBSTRING_INDEX(Proveedor, ' ', 1)
FROM productos;
```

Ejemplo 8

Ejemplo con funciones

Obtener el código de los productos pero con el orden de los caracteres invertido.

```
SELECT CodigoProducto, REVERSE(CodigoProducto)
FROM productos;
```

Ejemplo 9

Ejemplo con funciones

Obtener un gráfico de barras para el stock de cada producto. Utilizaremos el carácter '|' por cada unidad de stock.

```
SELECT Codigoproducto, REPEAT('|', CantidadEnStock)
FROM productos;
```

Ejemplo 10

Ejemplo con funciones

Obtener el email de cada empleado teniendo en cuenta que el usuario es su nombre en minúsculas y el dominio `@iesdoctorbalmis.com`.

```
SELECT CONCAT(LCASE(nombre), '@iesdoctorbalmis.com') AS email
FROM empleados;
```

Ejemplo 11

Ejemplo con funciones

Obtener la abreviatura del nombre y primer apellido de los empleados concatenando la inicial del nombre y la inicial del apellido1.

```
SELECT CONCAT(LEFT(Nombre, 1), LEFT(Apellido1, 1)) AS inicial
FROM empleados;
```

Funciones numéricas

Ejemplo 12

Ejemplo con funciones

Obtener un número aleatorio entre 0 y 9.

```
SELECT FLOOR(10 * RAND());
```

Funciones de fechas

Ejemplo 13

Ejemplo con funciones

Mostrar en campos diferentes el año, el mes y el día de la fecha de los pedidos.

```
SELECT YEAR(FechaPedido), MONTH(FechaPedido), DAY(FechaPedido)
FROM pedidos;
```

Ejemplo 14

Ejemplo con funciones

Mostrar todos los campos de pedidos realizados en enero del 2009.

```
SELECT *  
FROM pedidos  
WHERE YEAR(FechaPedido) = 2009 AND MONTH(FechaPedido) = 1;
```

Ejemplo 15

Ejemplo con funciones

Mostrar todos los campos de pedidos realizados en lunes.

```
SELECT *  
FROM pedidos  
WHERE DAYOFWEEK(FechaEntrega) = 2;
```

Funciones numéricas con varios registros

Ejemplo 16

Ejemplo con funciones

Mostrar el número total de pagos que se han recibido.

```
SELECT COUNT(*)  
FROM pagos;
```

Ejemplo 17

Ejemplo con funciones

Mostrar la cantidad en euros total que nos han ingresado.

```
SELECT SUM(Cantidad)  
FROM pagos;
```

Ejemplo 18

Ejemplo con funciones

Mostrar el importe del pago más pequeño y el importe de pago más grande recibido.

```
SELECT MIN(Cantidad), MAX(Cantidad)
FROM pagos;
```

Consultas **SELECT** de AGRUPACIÓN

Consultas de agrupación

Cuando necesitamos agrupar varios registros para realizar operaciones para sumar, contar, o calcular la media, el mínimo o el máximo, necesitaremos realizar una instrucción **SELECT** indicando qué registros agrupamos, es decir, qué campos mostramos de los registros comunes y sobre qué campos realizamos la operación.

La sintaxis para realizar una consulta agrupada a una tabla es:

```
SELECT [DISTINCT] campos
[FROM table_references]
[WHERE where_definition]
[GROUP BY {col_name | expr | position} [ASC | DESC], ... ]
[HAVING where_definition]
[ORDER BY {col_name | expr | position} [ASC | DESC] , ...]
[LIMIT [offset,] row_count]
```

Importante

El orden de las cláusulas, **obligatoriamente**, es:

SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER ... LIMIT

Ejemplos de las primeras consultas agrupadas

Con XAMPP-MySQL importar la **BD de jardinería** que facilitará el profesor.

Ejemplo 1

Contar

Mostrar el número de clientes (nombrar el nuevo campo **NumClientes**) que tenemos en cada **ciudad** .

```
SELECT Ciudad, COUNT(*) AS NumClientes
FROM clientes
GROUP BY Ciudad;
```

Ejemplo 2

Sumar operaciones numéricas

Mostrar el pedido y el importe total (**ImpTotal**) de todas las líneas de cada pedido. En la tabla detallepedidos tenemos el **CodigoPedido** , la **Cantidad** y el **PrecioUnidad** de cada artículo del pedido. Deberemos sumar **Cantidad * PrecioUnidad** de cada línea y luego sumarlas todas.

```
-- Sin Agrupar
SELECT CodigoPedido, Cantidad * PrecioUnidad AS ImpLinea
FROM detallepedidos;

-- Agrupando por Pedido
SELECT CodigoPedido, SUM(Cantidad * PrecioUnidad) AS ImpTotal
FROM detallepedidos
GROUP BY CodigoPedido;
```

Podemos añadir también filtros que deban cumplir los registros mediante condiciones en la cláusula **WHERE** .

Ejemplo 3

Agrupaciones con condiciones **WHERE**

Mostrar el número de artículos (**NumArticulos**) de cada Gama cuyas **PrecioVenta** mayor que 20€.

```
SELECT Gama, COUNT(*) AS NumArticulos
FROM productos
WHERE PrecioVenta > 20
GROUP BY Gama;
```

Pero puede ser que la condición a cumplir sea sobre los campos calculados. En estos casos, la condición irá en la cláusula **HAVING** .

Ejemplo 4

Agrupaciones con condiciones **HAVING**

Mostrar las gamas (**Gama**) de artículos que tengan más de 100 diferentes. Mostrar también el número de artículos (**NumArticulos**).

```
SELECT Gama, COUNT(*) AS NumArticulos
FROM productos
GROUP BY Gama
HAVING COUNT(*) > 100;
```

Incluso podemos tener consultas que combinen **WHERE** y **HAVING** simultáneamente.

Ejemplo 5

Agrupaciones con condiciones **WHERE** y **HAVING**

Mostrar los clientes que hayan realizado más de 1 pago de cantidad superior a los 3000€.

```
SELECT CodigoCliente, COUNT(*) AS NumPagos
FROM pagos
WHERE Cantidad > 3000
GROUP BY CodigoCliente
HAVING COUNT(*) > 1;
```

Consultas SELECT multitable

Consultas multitable de cruce (relación 1-N)

Las consultas multitable nos van a permitir procesar información de varias tablas conjuntamente. La sintaxis es la misma que hemos visto anteriormente para la sintaxis **SELECT**, pero en la cláusula **FROM** tendremos **varias tablas**.

Ejemplo 1

SELECT multitable

Mostrar los valores de la tabla **pagos** añadiendo el campo **NombreCliente**.

```
SELECT clientes.NombreCliente, pagos.*
FROM pagos, clientes
WHERE pagos.CodigoCliente = clientes.CodigoCliente;
```

- ¿Cuántos registros tienen la tabla pagos?
- ¿Cuántos registros tienen la tabla clientes?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición **WHERE**, ¿cuántos registros obtenemos? Justifícalo.

En este primer ejemplo se puede observar tanto en la cláusula **SELECT** como en la **WHERE** que para referirse a un campo de una tabla utilizamos la sintaxis: **tabla.campo**.

Por economía del lenguaje, podemos definir alias de tablas y utilizar estos para referirse a los campos como: **alias.campo**.

En una consulta multitabla no es necesario utilizar el prefijo de tabla (o alias) para referirse a un campo siempre y cuando no exista un campo con el mismo nombre en más de una de las tablas seleccionadas. En las consultas sobre una tabla no es necesario ni utilizar el nombre de la tabla ni un alias de esta para referirse a los campos de la tabla. No obstante, en muchos entornos de trabajo es ventajoso utilizar el nombre de tabla o alias ya que, en este caso, nos ofrece la opción de autocompletar y selección de campos. *PhpMyAdmin* nos brinda esa posibilidad.

A continuación veamos una variante del primer ejemplo con el fin de aclarar estos conceptos.

Ejemplo 1b

SELECT multitabla

Mostrar los valores de la tabla **pagos** añadiendo el campo **NombreCliente**. Utilizar alias a tablas y no utilizar prefijo de alias si no es necesario.

```
SELECT NombreCliente, p.*
FROM pagos p, clientes c
WHERE p.CodigoCliente = c.CodigoCliente;
```

Hemos utilizado **p** como alias de la tabla **pagos** y **c** como alias de la tabla **cliente**. Ahora accedemos a los campos de la tabla mediante el alias y no con el nombre de la tabla. La consulta se observa que:

- Con **p.*** seleccionamos todos los campos de la tabla pagos.
- En la condición **es necesario utilizar el alias** puesto que tanto en la tabla pagos como clientes hay un campo que se llama **CodigoCliente**.
- No hace falta utilizar un alias para acceder al campo **NombreCliente** puesto que sólo existe en la tabla clientes.

Ejemplo 2

SELECT multitabla

Mostrar los valores de la tabla **pedidos** añadiendo el campo **NombreCliente**.

```
SELECT clientes.NombreCliente, pedidos.*
FROM pedidos, clientes
WHERE pedidos.CodigoCliente = clientes.CodigoCliente;
```

- ¿Cuántos registros tienen la tabla pedidos?

- ¿Cuántos registros tienen la tabla clientes?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición **WHERE** , ¿cuántos registros obtenemos? Justifícalo.

Ejemplo 3

SELECT multitabla

Mostrar los valores de la tabla **empleados** añadiendo los campos **Ciudad** , **Pais** y la oficina del país **CodigoOficina** .

```
SELECT oficinas.Ciudad, oficinas.Pais, empleados.*
FROM oficinas, empleados
WHERE oficinas.CodigoOficina = empleados.CodigoOficina;
```

- ¿Cuántos registros tienen la tabla oficinas?
- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición **WHERE** , ¿cuántos registros obtenemos? Justifícalo.

En relaciones reflexivas, debemos cruzar una tabla consigo misma. Para poder hacer esto, tenemos que asignar un alias a cada tabla para diferenciarlas.

Veamos un ejemplo.

Ejemplo 4

SELECT multitabla con relación reflexiva

Mostrar los valores de la tabla **empleados** añadiendo los campos **Nombre** y **Apellido1** de su jefe.

```
SELECT jefes.Nombre, jefes.Apellido1, trabajadores.*
FROM empleados as trabajadores, empleados as jefes
WHERE trabajadores.CodigoJefe = jefes.CodigoEmpleado;
```

- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición **WHERE** , ¿cuántos registros obtenemos? Justifícalo.

Ejemplo 5

SELECT multitabla con varias tablas

Mostrar los siguientes valores:

- De **pedidos**: `CodigoPedido` y `FechaPedido`
- De **detallepedidos**: `CodigoProducto` y `Cantidad`
- De **productos**: `Nombre` y `Gama`

```
SELECT pedidos.CodigoPedido, pedidos.FechaPedido, detallepedidos.CodigoProducto, detallepedidos.Cantidad, productos.Nombre
FROM pedidos, detallepedidos, productos
WHERE pedidos.CodigoPedido = detallepedidos.CodigoPedido AND detallepedidos.CodigoProducto = productos.CodigoProducto
```

- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición `WHERE`, ¿cuántos registros obtenemos? Justifícalo.

Se puede observar que cuando se muestran datos de la tabla padre de relaciones con cardinalidad 1-N, estos se repiten ya que pueden tener muchos hijos. En el ejemplo anterior se puede ver claramente que los datos de la tabla pedidos se repiten tantas veces como líneas de detalle tengan.

Consultas multitable mediante INTERSECCIÓN

Las consultas de intersección se pueden ejecutar con la cláusula:

```
tabla1 INNER JOIN tabla2 ON condicion
```

El ejemplo anterior quedaría:

Ejemplo 1

SELECT multitable con INNER JOIN

Mostrar los valores de la tabla **pagos** añadiendo el campo `NombreCliente`.

```
SELECT clientes.NombreCliente, pagos.*
FROM (pagos INNER JOIN clientes ON pagos.CodigoCliente = clientes.CodigoCliente);
```

Veamos otro ejemplo de consulta multitable:

Ejemplo 2

SELECT multitable con INNER JOIN

Mostrar los valores de la tabla **detallepedidos** pero añadiendo los campos `FechaPedido` y `Estado` de la tabla pedidos. Comprueba que el resultado contiene el mismo número de registros que detallepedidos.

```
SELECT pedidos.FechaPedido, pedidos.Estado, detallepedidos.*
FROM (detallepedidos INNER JOIN pedidos ON detallepedidos.CodigoPedido = pedidos.CodigoPedido);
```

En las consultas multitabla mediante intersección, **es posible que haya registros que no se muestren por no haber cruce entre ellos**. Por ejemplo, si queremos saber el número de pedidos realizados por cada cliente, podríamos pensar en realizar la siguiente consulta:

Ejemplo 3

SELECT multitabla con INNER JOIN

Mostrar el número de pedidos realizados por cada cliente.

```
SELECT clientes.CodigoCliente, clientes.NombreCliente, COUNT(pedidos.CodigoPedido)
FROM clientes INNER JOIN pedidos ON clientes.CodigoCliente = pedidos.CodigoCliente
GROUP BY clientes.CodigoCliente, clientes.NombreCliente;
```

En la tabla de clientes hay 36 registros, pero el resultado de la consulta sólo muestra 19. Esto ocurre porque hay clientes que no han realizado todavía ningún pedido.

Para evitar esto, se introduce un cambio en la consulta que permite incluir todos los registros de una tabla de la intersección, que en nuestro caso es clientes. Lo haremos con la cláusula **LEFT** de la relación **clientes-pedidos**:

Ejemplo 4

SELECT multitabla con LEFT JOIN

Mostrar el número de pedidos realizados por cada cliente, incluyendo aquellos que no han realizado ninguno.

```
SELECT clientes.CodigoCliente, clientes.NombreCliente, COUNT(pedidos.CodigoPedido)
FROM clientes LEFT JOIN pedidos ON clientes.CodigoCliente = pedidos.CodigoCliente
GROUP BY clientes.CodigoCliente, clientes.NombreCliente;
```

Veamos otro ejemplo:

Ejemplo 5

SELECT multitabla con LEFT JOIN

Mostrar el **CodigoProducto** y nombre de la tabla **productos** junto con la suma de la cantidad

(**SumCantidad**) pedida en todos los pedidos existentes.



Tened en cuenta que de los productos que no haya habido ningún pedido debe aparecer 0.

```
SELECT productos.CodigoProducto, productos.Nombre, SUM(detallepedidos.Cantidad) AS SumCantidad
FROM productos LEFT JOIN detallepedidos ON productos.CodigoProducto = detallepedidos.CodigoProducto
GROUP BY productos.CodigoProducto, productos.Nombre;
```

La cláusula **RIGHT** permitiría que se mostrarán los de la segunda tabla en la relación, en vez de **LEFT** que muestra la primera.

Consultas multitabla mediante **UNION**

En ocasiones necesitamos unir el resultado de dos consultas. Para ello el resultado debe mostrar los mismo campos y con los mismos tipos de datos.

Por ejemplo, tenemos por un lado los clientes que han realizado pagos posteriores al 31/01/2009:

```
SELECT DISTINCT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31';
```

Por otra parte, tenemos los clientes que han realizado pedidos posteriores al 31/03/2009:

```
SELECT DISTINCT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

Los clientes que están en las dos consultas son 16, 23 y 27, pero hay clientes que han realizado pagos y no pedidos, y viceversa.



Ejemplo 1

SELECT multitabla con **UNION**

Mostrar el **CodigoCliente** de los clientes que han realizado pagos posteriores al 31/03/2009 o pedidos posteriores al 31/03/2009.

```
SELECT DISTINCT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31'
UNION
SELECT DISTINCT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

Consultas multitabla mediante SUBCONSULTAS

Una **subconsulta** es una instrucción **SELECT** que se usa dentro de otra instrucción **SELECT**.

Una **subconsulta** será **correlacionada** si aparecen datos de la consulta, es decir, si no se puede ejecutar de forma independiente.

Existen varias situaciones donde usaremos subconsultas. A continuación veremos algunos ejemplos.

Subconsultas en **WHERE** con operador de comparación

Para el siguiente ejemplo, primero buscamos la cantidad media que pagan los clientes.

```
SELECT AVG(Cantidad) AS CantidadMedia
FROM pagos;
```

Ejemplo 1

Subconsulta en **WHERE** con operador de comparación

Mostrar los registros de pagos que tengan cantidades superiores a la media.

```
SELECT *
FROM pagos
WHERE Cantidad > (SELECT AVG(Cantidad) AS CantidadMedia FROM pagos);
```

Subconsultas en **WHERE** con operador **IN**

El operador **IN** devuelve verdadero si el valor del campo de un registro está en el conjunto de valores devuelto por la subconsulta.

Ejemplo 2

Subconsulta en **WHERE** con operador **IN**

Mostrar la **Gama** de los productos de los que se haya pedido más de 30 unidades.

```
SELECT DISTINCT Gama
FROM productos
WHERE CodigoProducto IN (SELECT CodigoProducto FROM detallepedidos WHERE Cantidad > 30);
```

También puede usarse de forma negativa con **NOT IN**

Subconsultas en **WHERE** con operador **EXISTS**

El operador **EXISTS** es verdadero si la subconsulta devuelve al menos un registro.

Ejemplo 3

Subconsulta en **WHERE** con operador **EXISTS**

Obtener los datos de los empleados que tengan algún cliente asignado.

```
SELECT *  
FROM empleados  
WHERE EXISTS (SELECT * FROM Clientes WHERE CodigoEmpleadoRepVentas = empleados.CodigoEmpleado);
```

También puede usarse de forma negativa con **NOT EXISTS**

Subconsultas en **FROM**

Podemos utilizar subconsultas como tablas y colocarlas en la cláusula **FROM**.

Ejemplo 4

Subconsulta en **FROM**

Aunque la siguiente consulta se puede obtener mediante una consulta de intersección típica, usaremos una subconsulta para probar su funcionamiento en **FROM** de forma sencilla. Las subconsultas de **FROM** deben tener un alias que signaremos con **AS**.

Mostrar los datos de los empleados que trabajen en oficinas de Madrid.

```
SELECT empleados.*  
FROM empleados, (SELECT * FROM oficinas WHERE Ciudad = 'Madrid') AS OficinasMadrid  
WHERE empleados.CodigoOficina = OficinasMadrid.CodigoOficina;
```

La consulta anterior sin usar subconsulta sería:

```
SELECT empleados.*  
FROM empleados, oficinas  
WHERE empleados.CodigoOficina = oficinas.CodigoOficina AND oficinas.Ciudad = 'Madrid';
```